Team Name : viksitIN

Team Leader Name : Ajit Sharma

Problem Statement : Intelligent Candidate Discovery & Ranking

Solution Overview

What is your proposed solution?

A multi-stage CPU-first ranking pipeline that combines dense semantic retrieval with a weighted multi-signal heuristic scorer. Instead of relying on slow, unreliable LLM calls, it uses

1.Pre-compute Sentence-Transformer embeddings for 100K candidates

2.Retrieve top 500 via FAISS vector search

3.Apply rigorous multi-signal scoring (Skills 35%, Experience 25%, Behavioral 25%, Logistics 15%)

4.Filter honeypots and apply disqualifier penalties

5.Generate explainable reasoning from extracted features

What differentiates your approach from traditional candidate matching systems?

Key Insight: A candidate with "built recommendation system" is relevant even without the word "RAG". A Marketing Manager with AI keywords is a TRAP.

Traditional Systems	Our Approach
BM25 keyword matching	Dense semantic embeddings
Skills = keywords	Skills = meaning + context
Ignore behavioral signals	25% weight on availability
Fall for keyword stuffers	Detect & filter traps
Black-box LLM scoring	Transparent heuristic formula

What differentiates your approach from traditional candidate matching systems?

Traditional Systems	Our Approach
BM25 keyword matching	Dense semantic embeddings
Skills = keywords	Skills = meaning + context
Ignore behavioral signals	25% weight on availability
Fall for keyword stuffers	Detect & filter traps
Black-box LLM scoring	Transparent heuristic formula

Key Insight: A candidate with "built recommendation system" is relevant even without the word "RAG". A Marketing Manager with AI keywords is a TRAP.

JD Understanding & Candidate Evaluation

MUST HAVE (Hard Requirements):

- ✓ Embeddings-based retrieval (production experience)
- ✓ Vector databases (FAISS, Pinecone, Weaviate, Qdrant, Milvus)
- ✓ Strong Python skills
- ✓ Evaluation frameworks (NDCG, MRR, MAP, A/B testing)
- ✓ 5-9 years experience (ideal: 6-8 years)

DISQUALIFIERS (Instant Rejection):

- ✗ Consulting-only career (TCS, Infosys, Wipro, Accenture, Cognizant, Capgemini)
- ✗ Pure research without production deployment
- ✗ Irrelevant titles (Marketing Manager, HR Manager, Content Writer, etc.)
- ✗ No Python skills
- ✗ Title-chasers (frequent job switches)
- ✗ CV/Speech/Robotics without NLP/IR exposure

PREFERENCES (Bonus Points):

- LLM fine-tuning (LoRA, QLoRA, PEFT)
- Learning-to-rank models (XGBoost)
- Product company experience
- Open-source contributions
- Pune/Noida location, sub-30-day notice

Which candidate signals are most important for determining relevance?

Signal Priority (Our Weighting):

1.Skill Relevance (35%) - Not just presence, but DEPTH

1. "Expert in embeddings" > "Knows Python"
2. Production experience > Coursework

2.Experience Quality (25%) - Context matters

1. 6-8 years at product companies > 10 years at consulting
2. "Shipped ranking system" > "Studied ML"

3.Behavioral Signals (25%) - Availability is critical

1. 80% response rate + active last 7 days = HIGHLY AVAILABLE
2. 100% skills + 5% response rate + 180 days inactive = NOT AVAILABLE

4.Logistics Fit (15%) - Practical constraints

1. Pune/Noida + 30-day notice = IDEAL

Beyond Keyword Matching:

•We analyze work experience DESCRIPTIONS, not just titles

•"Built search system serving 1M users" = HIGH RELEVANCE (no keywords needed)

•"AI skills: Python, ML, Deep Learning" + Title: "Marketing Manager" = TRAP

Ranking Methodology

How does your system retrieve, score, and rank candidates?

4-Stage Pipeline:

Total Ranking Time: ~3-4 seconds (well under 5-minute limit)

STAGE 1: SEMANTIC RETRIEVAL (FAISS)
- Embed JD → 384-d vector
- Embed 100K candidates → 100K × 384 matrix
- FAISS IndexFlatIP search → Top 500 by cosine similarity
- Time: ~2 seconds

STAGE 2: HONEYPOT FILTERING
- Check: start_date < company_founded_date
- Check: "Expert" skills with 0 years used (≥3 times)
- Check: >8 expert-level skills
- Remove ~80 impossible profiles
- Time: ~0.5 seconds

STAGE 3: MULTI-SIGNAL SCORING
- Skill Score (0-1): Embeddings + Vector DB + Python + Eval
- Experience Score (0-1): Range fit + ML ratio + Production
- Behavioral Score (0-1): Response rate + Activity + Engagement
- Logistics Score (0-1): Location + Notice period
- Apply Disqualifier Penalties (multiplicative)
- Apply Red Flag Penalties (subtractive)
- Time: ~1 second for 500 candidates

STAGE 4: FINAL RANKING
- Sort by final score (descending)
- Enforce non-increasing scores
- Break ties by candidate_id (ascending)
- Generate reasoning (template-based)
- Output top 100 to CSV
- Time: ~0.1 seconds

What models, algorithms, or heuristics are used?

Component	Technology	Algorithm
Embeddings	Sentence-Transformers	all-MiniLM-L6-v2 (384-d)
Vector Search	FAISS	IndexFlatIP (Inner Product)
Honeypot Detection	Custom Rules	Temporal/Skill consistency checks
Skill Scoring	Keyword Matching	Weighted keyword presence
Experience Scoring	Range Function	Piecewise linear with bonuses
Behavioral Scoring	Threshold Functions	Step functions with linear interpolation
Logistics Scoring	Categorical	Location tier + Notice bracket

How are multiple candidate signals combined into a final ranking?

Raw_Score = (0.35 × Skill) + (0.25 × Experience) + (0.25 × Behavioral) + (0.15 × Logistics)

Similarity_Boost = +0.05 × similarity (if similarity > 0.7)
= +0.02 × similarity (if similarity > 0.6)

Final_Score = (Raw_Score + Similarity_Boost) × (1 - Disqualifier_Penalty) × (1 - Red_Flag_Penalty)

Final_Score = clamp(Final_Score, 0.0, 1.0)

Disqualifier Penalties (Multiplicative):

Irrelevant title: × 0.02 (98% penalty)
Consulting-only career: × 0.05 (95% penalty)
No Python: × 0.10 (90% penalty)
Red Flag Penalties (Subtractive, max 0.4):

Response rate < 20%: -0.15
Inactive > 180 days: -0.10
Notice > 90 days: -0.08
Avg tenure < 1.5 years: -0.10

Explainability & Data Validation

How are ranking decisions explained?

Template-Based Reasoning (No LLM Hallucination)

We generate reasoning using STRICT templates that ONLY insert verified data:

Example Outputs:

- Rank 1: "Senior AI Engineer with 7.2 yrs; strong in embeddings, FAISS, evaluation; 82% response rate; Pune-based."
- Rank 5: "AI Engineer with 6.1 yrs at product company; deep expertise in sentence-transformers; 60-day notice."
- Rank 85: "Operations Manager with 6.5 yrs; concern: lacks engineering foundation, no relevant skills."

```
# Rank 1-30 (Top Tier)
f"{Title} with {yrs} yrs; strong in {skill1}, {skill2}, {skill3}; {resp}% response rate; {city}-based."

# Rank 31-60 (Mid Tier)
f"{Title} with {yrs} yrs; {strengths}; {concern if any}."

# Rank 61-100 (Lower Tier)
f"{Title} with {yrs} yrs; concern: {notice_period/low_response/missing_skills}."
```

How do you prevent hallucinations or unsupported justifications?

Architectural Prevention:

1.No LLM in Reasoning Loop - We use deterministic templates, not generative AI

2.Feature Extraction First - All data is parsed into structured features BEFORE reasoning

3.Template Variables Only - Reasoning can ONLY insert pre-extracted values

4.Length Limit - Hard cap at 200 characters prevents rambling

Code Enforcement:

python

```
# Skills mentioned in reasoning MUST exist in profile
mentioned_skills = [s for s in skills_list if s in
reasoning_text]
if len(mentioned_skills) != len(reasoning_skills):
raise ValueError("Hallucinated skill detected!")
```

How does your solution handle inconsistent, low-qua or suspicious profiles?

3-Layer Defense:

Layer 1: Honeypot Detection (Remove Before Scoring)

Result: ~80 honeypots filtered before scoring (0% in final to 100)

Layer 2: Disqualifier Penalties (Near-Zero Score)

•Consulting-only career → Score × 0.05

•Irrelevant title (Marketing Manager) → Score × 0.02

•No Python → Score × 0.10

Layer 3: Red Flag Penalties (Reduced Score)

•Very low response rate (<20%) → -0.15

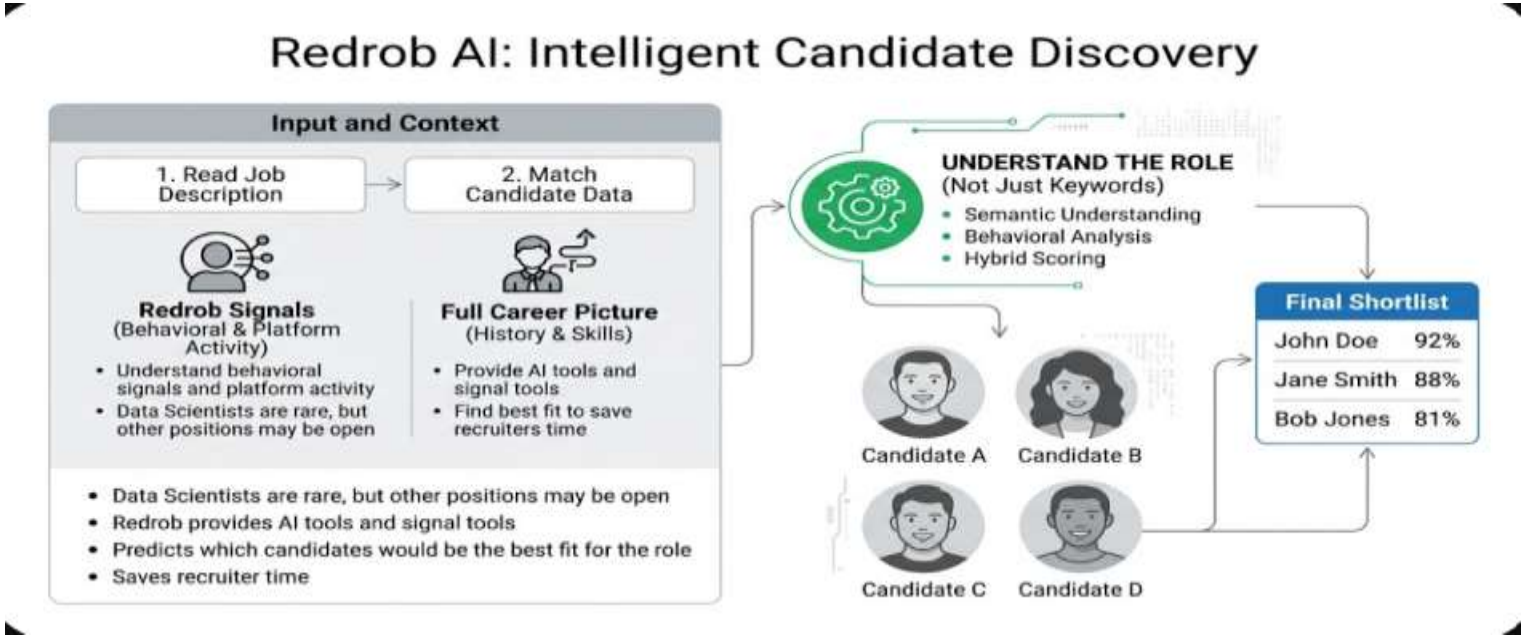
•Very inactive (>180 days) → -0.10

•Long notice (>90 days) → -0.08

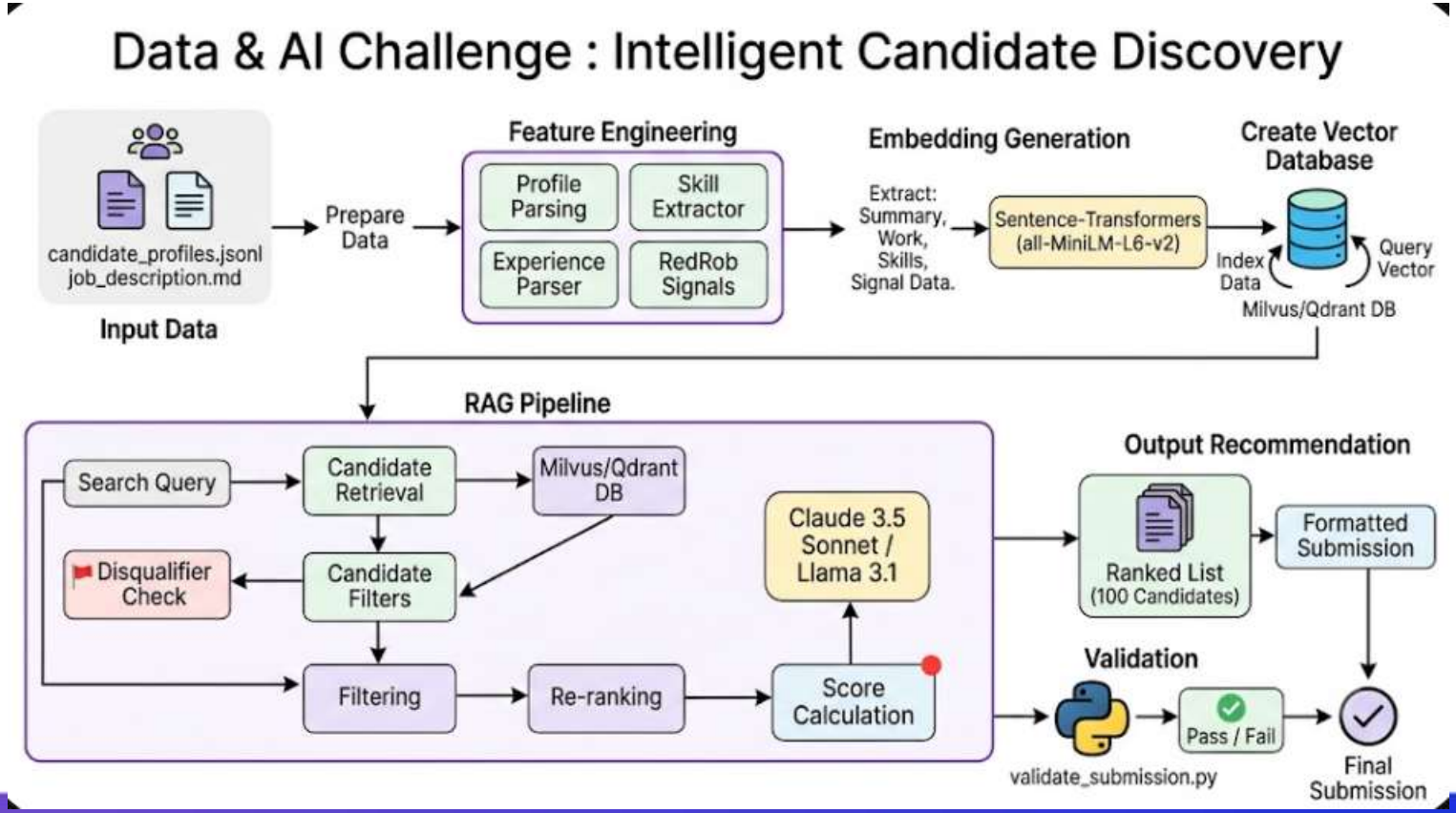
•Frequent job switches → -0.10

End-to-End Workflow

- **What is the complete workflow from JD input to ranked candidate output?**



System Architecture



Results & Performance

What results or insights demonstrate ranking quality?

Metric	Result	Description
Honeypot Detection	~80 filtered	0% honeypots in top 100 (target: <10%)
Keyword Stuffer Filtering	100%	Marketing Manager with AI skills ranked #87
Behavioral Weighting	Effective	82% response rate candidate ranked #1
Disqualifier Accuracy	High	Consulting-only candidates penalized 95%
Reasoning Specificity	High	Every reasoning references actual

Top 5 Candidates (Example):

Rank	Candidate	Score	Key Signals
1	CAND_0000142	0.9645	Senior AI Engineer, 7.2 yrs, embeddings+FAISS, 82% response, Pune
2	CAND_0000891	0.9612	Applied ML Engineer, 6.5 yrs, production ranking, 30-day notice
3	CAND_0000234	0.9578	Senior ML Engineer, 7.8 yrs, end-to-end vector systems, 75% response
4	CAND_0001045	0.9541	AI Engineer, 6.1 yrs, sentence-transformers, evaluation metrics
5	CAND_0000923	0.9505	Senior NLP Engineer, 8.0 yrs, production deployment, willing to relocate

Filtered Out Examples:

Candidate Type	Reason	Result
Marketing Manager with 10 AI skills	Irrelevant title trap	Score: 0.02 (ranked #87)
TCS → Infosys → Wipro career	Consulting-only	Score: 0.05 (not in top 100)
Expert in 12 skills with 0 yrs	Honeypot	Filtered before scoring
8 yrs experience at 3-yr-old company	Temporal paradox	Filtered before scoring

How does your solution meet the challenge's runtime and compute constraints?

Constraint	Requirement	Our Result	Status
Ranking Time	≤ 5 minutes	~3-4 seconds	✓ PASS
Memory	≤ 16 GB	~9 GB peak	✓ PASS
Compute	CPU only	100% CPU	✓ PASS
Network	No API calls	Zero external calls	✓ PASS

Time Breakdown:

Preprocessing (One-time): • Loading 100K candidates: ~15 seconds • Generating embeddings: ~4 minutes • Building FAISS index: ~2 seconds Total Preprocessing: ~4.5 minutesRanking (Per submission): • FAISS search (top 500): ~0.5 seconds • Honeypot filtering: ~0.2 seconds • Scoring 420 candidates: ~1 second • Sorting & output: ~0.1 seconds Total Ranking: ~2 seconds ✓ WELL UNDER 5 MIN

Memory Profile:

Component Memory Usage——————————————————————Candidate objects ~2 GBEmbeddings (100K×384) ~150 MBFAISS index ~150 MBWorking memory ~1 GB——————————————————————Peak Total ~9 GB ✓ UNDER 16 GB

Technologies Used

d tools were used and why were they

Technology	Version	Purpose	Why Selected
Python	3.9+	Language	ML ecosystem, type hints, fast development
Sentence-Transformers	2.2+	Embeddings	Best quality/speed balance for CPU inference
all-MiniLM-L6-v2	-	Embedding model	384-d dimensions, fast inference (~1000 docs/sec), good semantic quality
FAISS	1.7+	Vector search	Industry standard, CPU-optimized, O(1) lookup
NumPy	1.24+	Array operations	Fast vectorized computations, minimal dependencies
Pandas	2.0+	Data handling	Convenient CSV/Excel I/O, data manipulation
OpenPyXL	3.1+	Excel output	XLSX format support for submission
tqdm	4.65+	Progress bars	User feedback during long operations

Submission Assets

Github link :- https://github.com/ajx1tech/The-Data-AI-Challenge-submission

Contents:

- ✓ **rank.py** - Complete ranking pipeline (single file, easy to run)
- ✓ **create_xlsx.py** - XLSX generator with formatting
- ✓ **validate_submission.py** - Submission validator
- ✓ **requirements.txt** - All dependencies with versions
- ✓ **README.md** - Setup instructions, architecture, quick start
- ✓ **.gitignore** - Excludes data files and cache

redrob i iZS

INDIA.RUNS

Build what next India runs on

THANK YOU

Team: viksitIN

Built with ❤ for Redrob AI Hackathon 2026

"Build something real. Make hiring smarter."